

Internationalization (i18n) & Localization (l10n)

Java Backend Interview Reference — Multi-Language Applications

Glossary: i18n vs l10n vs g11n

Term	Full Name	Meaning
i18n	Internationalization	Designing software so it CAN be adapted for multiple locales — no hard-coded strings, flexible layouts, proper encoding
l10n	Localization	Adapting software FOR a specific locale — translating messages, formatting dates/numbers/currency, right-to-left support
g11n	Globalization	Business process of selling/operating globally — combines i18n + l10n + market strategy
Locale	Language + Region	e.g., en_US (English, USA), fr_FR (French, France), ar_SA (Arabic, Saudi Arabia). Controls language, date format, currency, number format
Translation	Message Bundles	Key-value files (properties, YAML, JSON) per locale that map message keys to translated strings
Fallback	Locale Fallback Chain	ar_SA -> ar -> default (en). If key missing in ar_SA, look in ar, then default bundle

Java i18n Core APIs

Locale

```
Locale l1 = Locale.forLanguageTag("fr-FR"); // IETF BCP 47 tag (preferred)
Locale l2 = new Locale("ar", "SA"); // legacy constructor
Locale l3 = Locale.getDefault(); // JVM default
Locale.Builder b = new Locale.Builder();
b.setLanguage("zh").setRegion("CN").setScript("Hans");
Locale l4 = b.build();
```

ResourceBundle — Message Bundles

```
// Resources: // messages.properties (default/English) // messages_fr.properties (French) //
messages_ar_SA.properties (Arabic, Saudi Arabia) // Load ResourceBundle bundle =
ResourceBundle.getBundle("messages", locale);
String greeting = bundle.getString("greeting"); //
messages.properties: greeting=Hello, {0}! // messages_fr.properties: greeting=Bonjour, {0}! // Format with
params: String msg = MessageFormat.format(bundle.getString("greeting"), userName);
```

MessageFormat — Plurals, Choices

```
// messages.properties: // items=There {0,choice,0#are no items|1#is one item|2<are {0} items}. String pattern
= bundle.getString("items");
MessageFormat mf = new MessageFormat(pattern, locale);
System.out.println(mf.format(new Object[]{3})); // "There are 3 items."
```

NumberFormat, DateTimeFormatter

```
// Number
NumberFormat nf = NumberFormat.getCurrencyInstance(Locale.FRANCE);
nf.format(12345.67); // "12 345,67 €"
// Date
DateTimeFormatter dtf = DateTimeFormatter.ofLocalizedDate(FormatStyle.LONG)
.withLocale(Locale.forLanguageTag("de-DE"));
LocalDate.now().format(dtf); // "22. Juni 2026"
```

Spring Boot i18n Setup

MessageSource Configuration

```
@Bean
public MessageSource messageSource() {
    ReloadableResourceBundleMessageSource ms = new
    ReloadableResourceBundleMessageSource();
    ms.setBasenames("classpath:messages/messages");
    ms.setDefaultEncoding("UTF-8");
    ms.setFallbackToSystemLocale(false); // don't use JVM locale as fallback
    ms.setCacheSeconds(3600); // reload every hour in dev
    return ms;
}

@Bean
public LocaleResolver localeResolver() {
    AcceptHeaderLocaleResolver r = new AcceptHeaderLocaleResolver();
    r.setDefaultLocale(Locale.ENGLISH);
    r.setSupportedLocales(List.of(Locale.ENGLISH, Locale.FRENCH,
    Locale.forLanguageTag("ar")));
    return r;
}

// Alternative: CookieLocaleResolver (persists locale in cookie)
// Alternative: SessionLocaleResolver (persists in HTTP session)
```

LocaleChangeInterceptor (query param)

```
@Configuration
public class WebConfig implements WebMvcConfigurer {
    @Override
    public void addInterceptors(InterceptorRegistry reg) {
        LocaleChangeInterceptor lci = new LocaleChangeInterceptor();
        lci.setParamName("lang"); // ?lang=fr
        reg.addInterceptor(lci);
    }
}
```

Using MessageSource in Service

```
@Service @RequiredArgsConstructor public class NotificationService { private final MessageSource
messageSource; public String getWelcomeMessage(String name, Locale locale) { return
messageSource.getMessage("welcome.message", new Object[]{name}, locale); } // welcome.message=Welcome, {0}!
(en) // welcome.message=Bienvenue, {0}! (fr)
```

Validation Messages

```
# ValidationMessages.properties javax.validation.constraints.NotBlank.message=This field is required
javax.validation.constraints.Size.message=Must be between {min} and {max} characters #
ValidationMessages_fr.properties javax.validation.constraints.NotBlank.message=Ce champ est obligatoire
```

Locale Detection Strategies

Strategy	How	Pros	Cons
Accept-Language header	Browser sends "Accept-Language: fr-FR,fr;q=0.9,en;q=0.8"	Automatic, no user action	Can be wrong if browser/device misconfigured
Query parameter	?lang=fr appended to URL	Easy to test, shareable links	Must be passed on every request
Cookie	lang=fr cookie set once, sent on all requests	Persistent across sessions	Cookie consent required in EU (GDPR)
User profile	User selects preferred language, stored in DB/profile	Most accurate, user-controlled	Requires authentication
Geolocation/IP	Infer country from IP, map to locale	Good first-time UX	VPN breaks it; inaccurate for language
Subdomain/path	fr.example.com or /fr/... routes	SEO-friendly, clear	Routing complexity, CDN setup needed

Message Bundle File Structure

```
src/main/resources/ messages/ messages.properties # default (English) messages_fr.properties # French (all
locales) messages_fr_CA.properties # French (Canada only – overrides fr where different)
messages_ar.properties # Arabic (RTL) messages_zh_CN.properties # Simplified Chinese messages_ja.properties #
Japanese messages_de.properties # German # messages.properties error.notfound=Resource not found
error.validation=Validation failed: {0} order.status.shipped=Your order #{0} has been shipped
order.total=Total: {0,number,currency} # messages_fr.properties error.notfound=Ressource introuvable
error.validation=Validation \u00e9chou\u00e9e: {0} order.status.shipped=Votre commande #{0} a \u00e9t\u00e9
exp\u00e9di\u00e9 order.total=Total: {0,number,currency}
```

RTL (Right-to-Left) Language Support

Aspect	Challenge	Solution
Text direction	Arabic, Hebrew, Urdu read right-to-left; mixing LTR text causes bidi issues	Use Unicode Bidi Algorithm; HTML: dir="rtl" on <html> or <body>; CSS: direction:rtl
API responses	Backend returns only text; frontend handles layout direction	Include "dir":"rtl" or "locale":"ar" in API response for frontend to use
Date/number format	Arabic Indic numerals differ from Western (1,2,3 vs ١,٢,٣)	Use NumberFormat/DateTimeFormatter with ar locale; client can choose numeral system
String sorting	Collation differs by language; Java String.compareTo is byte-based	Use java.text.Collator: Collator.getInstance(locale).compareTo(a,b)
Input validation	Name/address patterns differ; regex for ASCII names wrong for Arabic/CJK	Use Unicode-aware patterns: \p{L} instead of [a-zA-Z]

Date, Time & Timezone Handling

```
// ALWAYS store UTC in database // Convert to user timezone only in response @RestController public class
OrderController { @GetMapping("/orders/{id}") public OrderDTO getOrder(@PathVariable Long id,
@RequestHeader(value="X-Timezone", defaultValue="UTC") String tz, @RequestHeader(value="Accept-Language",
```

```
defaultValue="en") String lang) { Order order = orderService.findById(id); ZoneId zone = ZoneId.of(tz); // "
Europe/Paris" Locale locale = Locale.forLanguageTag(lang); // Format date in user's timezone and
localeZonedDateTime userTime = order.getCreatedAt().atZone(zone); String formatted = DateTimeFormatter .
ofLocalizedDateTime(FormatStyle.MEDIUM) .withLocale(locale) .format(userTime); return new
OrderDTO(order,formatted); } }
```

Rule	Detail
Store UTC	All timestamps in DB stored as UTC. Never store local times.
User timezone	Accept timezone via header (X-Timezone), JWT claim, or user profile. Use ZoneId.of(tz) with try/catch for invalid zones.
DST awareness	ZonedDateTime handles DST; Instant does not. Use ZonedDateTime for display, Instant for storage/calculation.
ISO 8601 in API	Return timestamps as "2026-06-22T14:30:00Z" (UTC, Z suffix). Frontend localizes.

Currency & Number Formatting

```
// Currency formatting respects locale conventions Locale usLocale = Locale.US; Locale frLocale =
Locale.FRANCE; Locale inLocale = new Locale("en", "IN"); NumberFormat usFmt =
NumberFormat.getCurrencyInstance(usLocale); NumberFormat frFmt = NumberFormat.getCurrencyInstance(frLocale);
NumberFormat inFmt = NumberFormat.getCurrencyInstance(inLocale); usFmt.format(12345.67); // "$12,345.67"
frFmt.format(12345.67); // "12\u00a0a0345,67\u00a0\u20ac" (12 345,67 €) inFmt.format(12345.67); // "
\u20b912,345.67" (₹12,345.67) // IMPORTANT: Store amounts as integers (cents/paise) to avoid float precision:
// 1234567 = $12,345.67 – divide by 100 for display only // Use BigDecimal for intermediate calculations
```

API Design for i18n

Pattern	Example	Notes
Accept-Language header	Accept-Language: fr-FR,fr;q=0.9	Standard HTTP; Spring reads automatically
Locale in response	{"locale":"fr-FR","data":{...}}	Helps frontend confirm which locale was used
Error messages localized	{"error":"Ressource introuvable","code":"NOT_FOUND"}	Always include code for programmatic use
Enum labels translated	{"status":"LIVRÉ","statusCode":"SHIPPED"}	Return both display label and stable code
Date in ISO 8601	"createdAt":"2026-06-22T14:30:00Z"	Frontend localizes per user timezone
Avoid locale in URL	/en/api/orders vs /api/orders?lang=en	Prefer headers; URL locale creates routing complexity

i18n Testing Strategies

Test Type	What to Check	Tool
Message key coverage	All keys in default bundle exist in all locale bundles	Custom test iterating ResourceBundle keys
Missing translation	Key present but value empty or same as default	CI script comparing bundle files
Pseudo-localization	Replace chars with accented versions: "Hello" → "Héllö". Reveals hard-coded strings and layout issues	Pseudolocale library or custom replacement
Date/number format test	Assert formatted output matches expected for each locale	JUnit + Locale.forLanguageTag
RTL layout test	Check layout mirrors correctly for Arabic/Hebrew	Browser automation with ar locale
Character encoding	Verify UTF-8 throughout: DB column, JVM, files, HTTP Content-Type	Check DB collation, file encoding, Spring response charset

Interview Q&A;

Q1: What is the difference between i18n and l10n?

i18n (internationalization) means designing the software to support multiple locales — externalizing strings, using proper date/number APIs, supporting Unicode. l10n (localization) means actually adapting the software for a specific locale — providing translated messages, formatting dates as per locale conventions. i18n is done by developers; l10n is often done by translators.

Q2: How does Spring Boot resolve the locale for a request?

Spring Boot uses a `LocaleResolver`. `AcceptHeaderLocaleResolver` reads the `Accept-Language` HTTP header. `CookieLocaleResolver` reads a locale cookie. `SessionLocaleResolver` reads from session. You configure a default locale (e.g., `Locale.ENGLISH`) as fallback. The `LocaleChangeInterceptor` lets users switch locale via a query parameter (`?lang=fr`).

Q3: How do you handle plurals in message bundles?

Use `MessageFormat` with `ChoiceFormat` pattern: `"items={0,choice,0#no items|1#one item|2<{0} items}"`. For complex pluralization rules (like Polish or Russian with 6 plural forms), consider ICU `MessageFormat` which follows CLDR plural rules, or a dedicated library like GNU `Gettext` with `PluralForms`.

Q4: Why should you never store local times in the database?

Daylight saving time transitions can cause ambiguous or skipped times. A time like "2024-03-10 02:30" doesn't exist in US Eastern because of spring-forward. Queries that compare or sort times break across DST boundaries. Always store UTC (as `TIMESTAMP WITH TIME ZONE` or as epoch millis), then convert to user timezone for display only.

Q5: How would you support Arabic (RTL) in your API responses?

The backend itself doesn't need to change direction — that's a frontend concern. However: (1) Translate all user-facing strings to Arabic in message bundles using UTF-8; (2) Use Arabic-locale `NumberFormat`/`DateTimeFormatter` for formatted values; (3) Include the locale/direction (`"dir":"rtl"`) in the response if frontend needs it; (4) Use Unicode-aware string operations (`Collator`, `\p{L}` patterns).

Q6: How do you test that all translation keys are present?

Write a JUnit test that loads the default bundle, iterates all keys, and for each supported locale verifies the same key exists in that locale's bundle. Fail the build if any key is missing or if the value equals the default (potential untranslated string). Run this in CI so missing translations break the build before release.

Q7: How would you dynamically add a new locale without redeploying?

Use `ReloadableResourceBundleMessageSource` with `setCacheSeconds` (e.g., 300) instead of `ResourceBundleMessageSource`. Store bundles in an external file system path or a database. When a new locale file is added, the source reloads it on the next cache expiry without restart. Alternatively, use a cloud-based translation platform (like Phrase or Lokalise) whose SDK polls for updates.

Q8: What is pseudo-localization and when do you use it?

Pseudo-localization replaces characters in the default locale with visually similar accented characters and wraps strings in brackets: "Hello, world!" becomes "[**é**ö, ö**d**!]**!**". This exposes hard-coded strings (they won't be transformed), layout issues (longer pseudo strings reveal truncation), and RTL problems early in development — before real translators are involved.